

Obsidian with Claude Code: a starting guide

Written for someone who has never connected the two. It teaches what good looks like and why, grounded in one real vault — `claude-memories`, 2996 notes, synced with Obsidian Sync, not a git repository until this week. It does not tour plugin menus. Generated from `useful-claude-add-ons@91669bf1` on 22 September 2026. Verify before trusting.

In one paragraph. An Obsidian vault is a folder of plain Markdown files, and Claude

Code can read and write that folder like any other. Everything that follows is about the

gap between "Claude touched a file" and "the change is actually safe" — how it got

there, whether anyone can undo it, and whether it belongs in the vault at all.

1. Two sync mechanisms, and why running both by habit is a risk

Obsidian Sync (the paid, official service) and git are not the same kind of protection, and treating them as interchangeable is the first mistake.

Question	Obsidian Sync	git (e.g. obsidian-git)
What it actually gives you	End-to-end encrypted sync between devices, with a rolling version history per file.	A full, inspectable history of every commit, and a remote copy wherever you push it.
How long history survives	Note versions: 1 month on the Standard plan, 12 months on Plus. Attachment versions: 2 weeks, on either plan.	Forever, or until someone rewrites history — nothing auto-expires.

Question	Obsidian Sync	git (e.g. obsidian-git)
What happens on conflict	Merges automatically; non-trivial conflicts can surface as a notice, not a guaranteed manual review.	A real merge conflict, marked in the file with <<<<<< markers, that blocks until a person resolves it.

Obsidian Sync's own retention numbers matter because they set a real deadline: a note edited badly and not caught within the plan's window cannot be recovered from Sync's history after that window closes. Git has no such clock.

obsidian-git's fire-into-the-void failure

The plugin's own name for its main action is "commit-and-sync," and that name is the trap. A commit is a purely local operation — it always succeeds, instantly, with nothing to fail on. The push to a remote is a separate network operation, and it is the one that can fail: no remote configured, an expired credential, an offline laptop, a rejected non-fast-forward push. Multiple issues against the plugin describe exactly this — commits that succeed while the push silently does not, sometimes with the failure notice itself suppressed by the very setting meant to reduce noise. The result: a vault whose owner believes it is "backed up by git," has a `.git`` folder full of real commits to prove it, and has no copy anywhere off that one machine, because every one of those commits fired into a void with no remote catching it.

What to actually check, not just set up once: that `git remote -v` names a real remote, that the last few commits show up on that remote (not just in the local log), and that a failed push produces a notification you will actually see rather than one a "reduce notifications" setting has quietly turned off. A 10-15 minute auto-commit interval is the plugin's own recommended default and is a reasonable choice for the commit half of the job — it says nothing about whether the push half is working.

Running Sync and git together

The two can coexist — many vaults do — but not carelessly. The concrete failure mode is in the `.obsidian` configuration folder: if both mechanisms are watching and writing to files like `workspace.json` or the git plugin's own `data.json`, they can fight over the same file. The accepted mitigation, from the plugin's own users, is to gitignore those files so only Sync manages them, and let git own the notes themselves.

2. How Claude actually reaches a vault

There are three ways to connect Claude Code to a vault, and they are not interchangeable in what they require to be running.

Route	What it needs	Watch for
Direct filesystem access	Nothing but a path. Claude reads and writes Markdown files exactly like it edits code.	No awareness of Obsidian's own state — a note open and unsaved in the app can be overwritten from underneath it.
Local REST API plugin	Obsidian itself running, with the plugin enabled. The bridge is an HTTP(S) server inside the running app , not a background daemon.	Close Obsidian, the bridge is gone — no error, just nothing listening on the port.
MCP server on top of the REST API	Everything the REST API needs, plus an MCP client (Claude Code) configured to trust its certificate.	The certificate trap below.

The certificate trap is real and it is documented by the plugin itself, not folklore: the Local REST API plugin generates its own certificate authority on first run and serves HTTPS from it. Testing that endpoint with `curl -k` proves nothing about whether an MCP client can connect — `-k` tells curl to skip certificate verification entirely, which is exactly the check an MCP client does not skip. A green `curl -k` result and a Node-based MCP client refusing to connect are both consistent with the same broken setup. The plugin's own fix is one of two things: trust its certificate authority (fetchable from the plugin's own HTTPS endpoint, then imported into the OS or the client's trust store), or fall back to its separate plain-HTTP endpoint if the client supports it. Confirm a connection by running the actual MCP tool call, not by curling the port.

Because the bridge lives inside the running application, "is the vault reachable" is really "is Obsidian open right now" — worth checking first whenever a vault tool call fails for no obvious reason.

3. What belongs in a vault, and what belongs in a repository

The instinct to put everything durable into the vault — including things generated from code — has a specific, dated counter-example worth citing, because it is a real decision this project made and then reversed.

The worked example. This repository's own CLAUDE.md records that a mechanical

code graph was once exported into an Obsidian vault, and the decision was reversed.

The stated reason: *"the map lived on one machine and reached nobody who cloned."* A

vault is one person's local application state — even synced, it is not something a

teammate who clones the repository can open. Anything that has to reach every clone

belongs in the repository as a tracked file, not in a vault, however convenient the

vault's linking and search are.

The general shape of the boundary that example implies:

Belongs in the vault

Notes about the work — decisions made, questions raised, what a session learned

Personal synthesis, second-order thinking, links between ideas

Material that is fine living on one synced set of devices

Belongs in the repository

Artefacts the work produces — code, generated diagrams, data files another clone needs to read

Anything a CI job, a teammate's clone, or a different machine has to see without asking one specific person

Material whose absence from git means "nobody but me has this"

4. Keeping a gardener queue from reaching 81

A "gardener" or inbox pattern — write everything into one place during a session, then distill it into real notes later — is a standard and sound practice. The failure is not the pattern; it is treating "later" as open-ended. There is no Obsidian-specific published cadence for this — the closest grounded guidance is the general inbox-processing practice from task management: process toward empty frequently (daily is the common recommendation) rather than batching, with a fixed weekly review as the backstop for whatever the daily pass missed. An inbox that goes unprocessed for 18 days does not fail gracefully — it grows, and every day added makes each entry harder to distill because the session context behind it fades.

Two practical anchors for a queue that has already grown large: don't try to clear it in one sitting by writing careful notes for all 81 entries — time-box a catch-up pass and treat anything not distilled in that window as a conscious decision to skip, logged as skipped rather than left silently unprocessed. And separate "capture" from "distill" as two different pieces of work with two different urgencies: capture (writing into the inbox) is cheap and can happen every session; distillation (turning that into a linked, dated note) is the part that should be scheduled, not squeezed in.

5. Maps of content

A map of content is a single curated note that links out to the notes on one topic — it is a navigation layer sitting above the notes, not a replacement for links inside them. The distinction that matters against the two mechanisms Obsidian already gives you for free:

Mechanism	How it groups notes	Limit
Folders	One folder per note, chosen at save time	A note can live in exactly one place, decided before you know how it will be used
Tags	Automatic membership by label	Flat — a tag says a note belongs to a topic, not how it relates to the other notes carrying the same tag
Maps of content	A hand-curated note, edited deliberately	Only exists where someone chose to write one, and goes stale if nobody maintains it

The reason a map earns the maintenance cost folders and tags don't ask for: a note can

appear on more than one map, or none, and a map can impose real structure — sub-headings, an order, a deliberate omission — that a folder or a tag cannot express. Treat a map as itself a kind of gardener output: it is worth building for an area of the vault once that area has enough notes that finding your way through search alone stops working, and it needs the same periodic revisit as any other note, or it quietly stops matching the vault it describes.

Sources for the claims above are listed in the research this guide was built from; ask for them if a specific figure needs to be re-verified. Two items are explicitly unconfirmed and were not made to look settled: whether Obsidian Sync and git-based sync running together is a documented incompatibility (found only as forum consensus, not official policy), and the exact note count at which Omniseach's index turns slow (its own docs say "several thousand files," with no single published number).